Generative AI Project

Bastien Rousseau, Mattia Segreto
barou1817@uis.no, maseg2421@uis.no

VAE Architecture

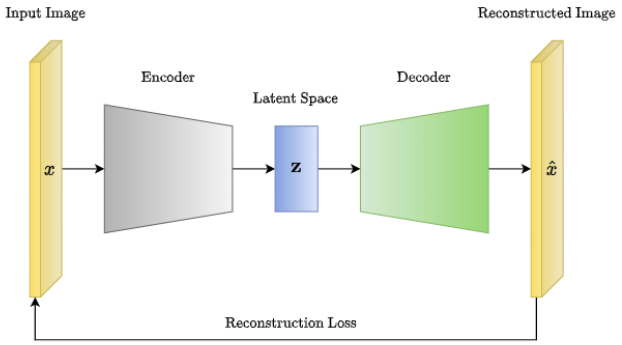

Figure 1. Schematic view of the VAE architecture [8]

GAN Architecture

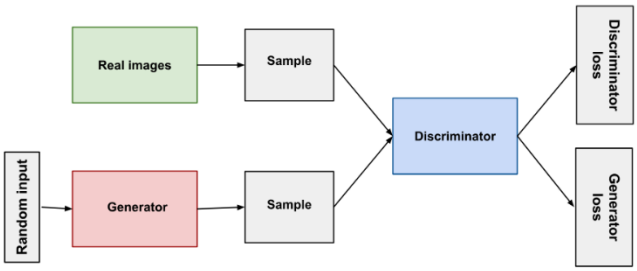

Figure 2. Schematic view of the GAN architecture [9]

1 – Introduction

Generative modeling is a field of machine learning enabling the synthesis of realistic images learned from probability distributions. The main architectures are, *Variational Autoencoders (VAEs)* and *Generative Adversarial Network (GANs)* or *Deep Convolutional Generative Adversarial Networks (DCGANs)*. These networks have demonstrated good capabilities for image generation [6][7]. These models rely on two different principles [4][6]. On the one hand, VAEs aim at maximize a lower bound on the data similarity through probabilistic latent representation (latent space). On the other hand GANs use adversarial training process between a generator and a discriminator to improve the quality of generated samples.

This report focuses on implementing and comparing these two approaches on the Fashion-MINST dataset. It is a benchmark composed of 70,000 grayscale images (28x28 pixels) of clothing items across 10 categories. The main goal is to evaluate and contrast the reconstruction and generation abilities of the models. This will be compared in term of visual quality, training stability, diversity of samples generated.

To ensure fairness, both models will be train under comparable experimental conditions such as similar convolutional architectures, data normalization, data regularization (to avoid overfitting).

With this analysis of both approaches, this report will highlight their respective strengths and weaknesses.

2 – Materials and Methods

2.1 Dataset and Preprocessing

The experiments will be conduct on the **Fashion-MNIST** dataset, composed of 60,000 training and 10,000 test images.

Each image is 28×28 grayscale and is normalized to [0,1] for the VAE pipeline and to [-1,1] for the GAN pipeline. To make sure of the reproducibility of the experiments and visualize the data, an exploratory data analysis (EDA) was first performed to investigate class balance, visualize representative samples, and also compute basic statistics.

All images were reshaped into tensors of size (28, 28, 1) and normalized to float values in [0,1] for VAE / [-1,1] for GAN. No class imbalance correction was necessary, as the original dataset is already evenly divided across every categories.

For a better understanding of the dimensionality of Fashion-MNIST, **Principal Component Analysis (PCA)** was led to the flattened pixel vectors (784-dimensional).
The PCA showed that:

- The **first two principal components** capture a significant portion of the variance (around 60%), revealing clear clusters which correspond to clothing categories.

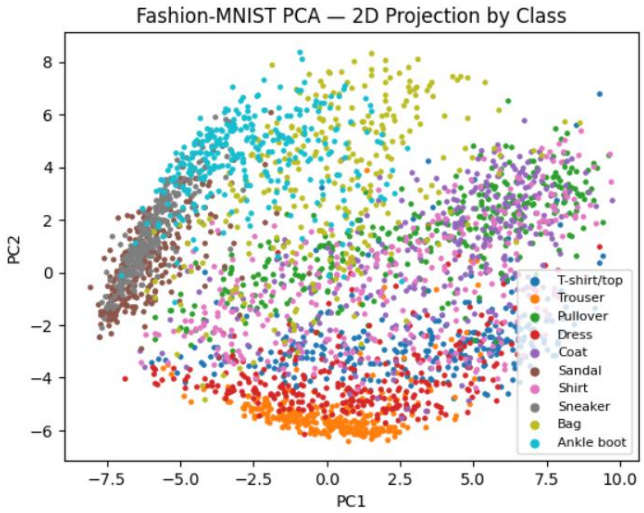

Figure 3. 2D projection of classes

- Plotting the samples in the 2D PCA space revealed partial separation between classes such as *Sneakers* and *Bags*. The categories which are more visually close (such as Coat and Pullover).

- The Cumulative Explained Variance revealed that approximately **40** components explain over **85%** of the total variance, confirming that Fashion-MNIST is a low-dimensional manifold. It is a property that generative models can exploit to be more efficient.

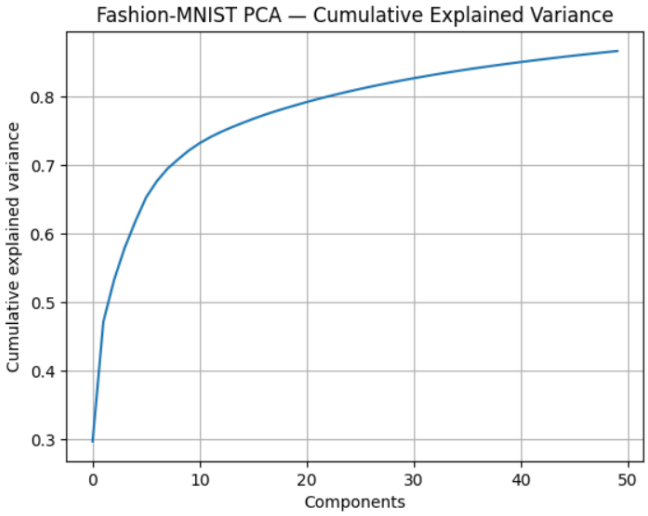

Figure 4. Cumulative Explained Variance based on number of component

This PCA analysis motivated us to use low-dimensional latent spaces (e.g., 32-dimensional for the VAE,128-dimensional input for the GAN). Indeed, these dimensions are sufficient to encode the major visual variations while avoiding unnecessary complexity.

2.2 Methods and Models

All models were implemented in **TensorFlow/Keras** and trained on GPU100.

2.2.1 Variational Autoencoder (VAE)

The VAE consists of three components [4]:

- **Encoder**: A neural network compressing the input image into a latent vector $z \in \mathbb{R}^{32}$ (32-dimensional vector) or $z \in \mathbb{R}^{64}$ (64-dimensional vector). It outputs both the mean μ and log-variance $\log \sigma^2$of the latent Gaussian distribution.
- **Sampling layer**: Implements the reparameterization trick $z = \mu + \sigma \cdot \varepsilon$, with $\varepsilon \sim \mathcal{N}(0, I)$, ensuring differentiability during backpropagation.
- **Decoder**: A symmetric transposed-convolutional network reconstructing the image from the latent representation constructed earlier.

The loss function is :

$$L = BCE(x,\hat{x}) + \beta \cdot D_{KL}(q(z|x)||p(z))$$

Where BCE stands for Binary Cross Entropy and D_{KL} is Kullback–Leibler divergence [4].

The **hyperparameter** β controls the balance between these two objectives [5]:

- A small β prioritizes reconstruction accuracy but may produce a disorganized latent space (the model will mostly just learn images).
- A large β enforces strong regularization, encouraging disentangled latent features but possibly degrading reconstruction quality(the model will introduce a blur during reconstruction).

Grid Search

A grid search is performed with the purpose of identify the VAE configuration that has a better deal between reconstruction quality and latent space regularization on the Fashion-MNIST dataset.

The Grid search focus is the validation loss and is objective function of the training. It's composed by reconstruction loss plus weighted KL divergence. The configuration which has been reached the lowest validation loss has been chosen as best-model. The modus operandi for each tested configuration follows these steps:

1. The model is trained on the Fashion-MNIST training set with a fixed validation split.
2. The training and validation losses are recorded at each epoch.
3. The configuration with the lowest validation loss at convergence is retained for qualitative evaluation and sample generation.

Several configuration were explored and mainly focus on those hyper parameters:

- Latent dimensionality (z_dim): determines the representational capacity of the latent space (32 or 64).
- KL weighting schedule: comparison between beta_mode="fixed" and beta_mode="anneal". When beta is in the anneal mode it is controlled by beta_init, beta_target, and beta_warmup otherwise those parameter are irrelevant
- Learning rate (lr): has a range between 5x10^-3 and 1x10^-4
- Dropout and weight decay: for generalization improvement moderate the regulation.
- Training setup: fixed batch=128, epochs=75, and shuffle=50000.

Evaluated configurations with the following format (name : key hyperparameters).

- vae_v1_base : z_dim=32, lr=1e-3, batch=128, epochs=75, dropout=0.10, weight_decay=1e-4, beta_mode="fixed", beta_init=1.0, beta_target=2.0, beta_warmup=0.
- vae_v2_beta_fixed : same as *v1* but with minor initialization and seed variations; baseline fixed-β reference.
- vae_v3_beta_anneal : z_dim=32, lr=8e-4, dropout=0.10, beta_mode="anneal", beta_init=0.0, beta_target=1.0, beta_warmup=40.
- vae_v4_z64 : z_dim=64, lr=5e-4, beta_mode="anneal", beta_init=0.0, beta_target=1.0, beta_warmup=40, other parameters as in *v3*.
- vae_v5_dropout012 : z_dim=32, lr=8e-4, dropout=0.12, beta_mode="fixed", beta_init=1.0, weight_decay=1e-4.

(Some intermediate variants or unstable runs that consistently produced inferior validation losses were excluded early to focus on the most promising settings.)

Conv-β-VAE(Convolutional + β VAE with KL ann.)

After the first grid search and training the results show some improvement margins. In order to achieve better results, a convolutional β-VAE (Conv-β-VAE) has been created and trained. This idea use two different idea to improve the results first it use convolutional encoder and decoder and second use the β-VAE loss [5][10].This choice has win against the Data Augmentation because like we will se in the results section the main problem of the synthetic sample was not that the model was overfitting but moreover that the sample seem a little bit blurred and this choice is more focus in mitigate this factor (there is already a lightweight data augmentation in the encoder so the choice is to not enhancing it and focus in this new version). The blurriness of the sample is a well-known problem in VAE synthetic sample generation and it is due to the loss minimization which tends to generate less sharp output.

Convolutional encoder and decoder will make it more efficient for images because it will allow the network to get local features. In addition, the loss function will be the β-VAE loss for a better control over KL divergence and encourage a more structured latent representation (a better diversity among the generated samples). Moreover, applying

KL annealing will help to stabilize the training and prevent posterior collapse.

2.2.2 Generative Adversarial Network (GAN)

The Generative Adversarial Network (GAN) is not looking for a likelihood function optimization instead the aim during the train is a two-player minimax game between a Generator and a Discriminator [6]. Both were trained together, and the aim of the generator is to produce, starting from a random vector, a real image that the discriminator will not be able to distinguish from a real image. On the other hand, the discriminator aims to distinguish a real image from the sample created by the generator. This adversaria process improves both the actors, making the generator more able to create realistic samples and the discriminator more able to distinguish between fake and real images.

The implementation of the GAN is in accordance with the Deep convolutional GAN (DCGAN) architecture, adapted to the Fashion-MNIST dataset. Both networks use **convolutional building blocks** designed to ensure stable gradient flow and to capture **spatial** structures [7].

The Generator takes as input a random **latent vector** $z \in R^{128}$, sampled from a standard normal distribution: $z \sim N(0,I)$

It then transforms this latent code through a sequence of learned **up sampling (transposed convolution)** operations, ultimately producing an image $G(z)$ that matches the format of the training data (a 28×28 grayscale image).

The **Generator** architecture has been structured using:

- A **dense layer** projects the latent vector z into a low-resolution feature map of size **7 × 7 × 256**, providing a learned starting point for spatial up sampling.
- **Batch Normalization** is applied to stabilize training and accelerate convergence by reducing internal covariate shift.
- **LeakyReLU** activations introduce non-linearity while preserving gradient flow, preventing dead neurons.
- A sequence of **Transposed Convolution (Conv2DTranspose)** layers progressively doubles the **spatial** resolution and reduces the number of feature maps at each step.
- The final **tanh activation** maps outputs to the range **[-1, 1]**, consistent with the input normalization of the training data.

This architecture enables the generator to learn **hierarchical representations** of textures and shapes while preserving **spatial coherence** across the generated images.

The **Discriminator** architecture has been structured in this way:

- A stack of **Convolutional layers** with increasing filter depths (64, 128, 256) and stride 2, progressively **down sampling** the input and extracting hierarchical spatial features.
- **LeakyReLU** activations are used to prevent neuron inactivity and maintain gradient flow for negative inputs.
- **Dropout** layers act as regularizers, reducing overfitting and improving generalization.
- A **Minibatch Standard Deviation** layer is added before the final dense layers to encourage diversity in generated samples and mitigate mode collapse.
- After **Flattening**, a **Dense(128)** layer captures intermediate feature representations, followed by a **Dense(1)** layer that outputs a **logit score** (no Sigmoid activation).

The discriminator's output represents how "real" or "fake" an input image appears, and the final Sigmoid transformation is implicitly handled by the loss function (Binary Cross Entropy (from_logits=True)).

Training Overview

Training a Generative Adversarial Network can be formulated as a **minimax optimization problem** between the Generator *G*G and the Discriminator *D*D [6]:

$$\underbrace{min}_{G}\ \underbrace{max}_{D}\ E_{x\sim p_{data}}[\log D(x)]+E_{z\sim pz}[\log(1-D(G(z)))]$$

Where *pdata* denotes the **training data distribution**, and *pz* is the **latent noise distribution**, typically a standard normal *N(0,I)*.

In practice, the **Discriminator** aims to **maximize** the above objective by increasing *D(x)* for real samples and decreasing *D(G(z))* for fake samples, while the **Generator** seeks to **minimize** it by producing images that push *D(G(z))* toward 1.

To improve gradient flow during training, the **non-saturating variant** of the generator loss is used:

$$L_D = -E_{x\sim p_{data}}[logD(x)] - E_{z\sim p_z}[log(1-D(G(z))]$$

$$L_G = -E_{z\sim p_z}[logD(G(z))]$$

Both networks are updated in **alternating steps**, ensuring that progress in one induces counter-adaptation in the other, leading to the adversarial learning dynamic.

Grid Search

The grid search aims is to find a trading configuration of GAN which is able to generate high quality sample optimizing the trade off of a good generator and a robust discriminator. This can be made exploring several configuration of hyper parameters during the training phase. Kernel Inception Distance (KID), that is a similarity measure of the real distribution and the generated one, is the target metric of this GRID search. The kid estimate the Maximum Mean Discrepancy (MMD) computed in the feature space which is extracted from a pretrained InceptionV3 network. So kid make a comparison between the extracted feature of the real and synthetic images and compute the MMD

The modus operandi for each tested model follows these steps:

- The generator creates 2,000 synthetic samples
- InceptionV3 network, features have been extracted from its average pooling layer starting from the Feature distribution the KID is computed.

Then, the model which achieve the lowest KID score is chosen. Focusing now on the tested configuration the main differences among the tested version can be found among this hyperparameters set:

- Batch size: 64 and 128 which asses the effect of gradient noise on training stability.
- TTUR (Two Time-Scale Update Rule): focuses on learning rates ratio of the generator and discriminator.
- Feature Matching Loss relative weight of the auxiliary loss used to mitigate *mode collapse* and encourage structurally coherent samples.
- Discriminator Regularization: presence or absence of *label smoothing* (e.g., real label = 0.9) and moderate *dropout* to reduce overfitting.
- Data Augmentation: which choose if use simple geometric transformations (flip, translation, *instance noise*) or DiffAugment that is a differentiable data augmentation strategy.

Evaluated configurations with the following format (name: key _hyperparameters).

- v1_base : BATCH_SIZE=128, LR_D=2e-4, LR_G=1e-4, β1=0.5, β2=0.999, λ_FM=10.0
- v2_bs64 : same as *v1* but with BATCH_SIZE=64
- v5_fm5 : same as *v1* but with λ_FM=5.0 (reduced feature matching weight)
- v6_ttur_smooth : BATCH_SIZE=64, LR_D=3e-4, LR_G=1.5e-4, β1=0.45, β2=0.999, *label smoothing = 0.9*, *dropout_D = 0.1*
- v7_fm15_aug : BATCH_SIZE=64, LR_D=2e-4, LR_G=1e-4, β1=0.5, β2=0.999, λ_FM=15.0, *DiffAugment* with policies ["translation", "color"], *dropout_D = 0.05*

(Configurations v3 and v4 were discarded early in the experiments, as they consistently produced inferior results compared to the others.)

Then the best configuration found was integrate with the Exponential Moving Average

Stabilization: Exponential Moving Average (EMA)

The GAN training is typically unstable because the two adversarial training and usually cause in the generator an oscillating behavior that could lead in a loss of image quality.

In order to mitigate this effect in order to optimize the best performing model of the grid search an Exponential Moving Average (EMA) of the generator's weights is used during the training. The purpose of this feature is to provide a smoothed shadow copy which stabilize the generator weight updating:

$$\theta_{EMA} \leftarrow \alpha\theta_{EMA} + (1-\alpha)\theta_G$$

Where, $\alpha \in [0,1]$.

This suppresses a rapid fluctuation of the parameters improving model stability during the training phase. During the inference this is usually lead in an higher quality and diverse samples.

In our experiments the EMA integration effectively takes these concrete advantages but we will see better in the results and discussion sections [12][13][14].

3 – Results

This section presents the experimental outcomes obtained from training and evaluating both the Variational Autoencoder (VAE) and the Generative Adversarial Network (GAN) on the Fashion-MNIST dataset.

3.1 – VAE training and reconstruction performance

The VAE training process converged smoothly, reflecting the stability of the reconstruction–regularization trade-off. Among the tested configurations, the second setup (index 0) achieved the lowest validation loss .

The reconstruction shows that the model has learn how to encode and decode images from the dataset. The shape of different classes are preserved even if there is some blurriness which is caused by the loss function (BCE part). This small degradation of the image is inevitable to be able to generalize and create completely new images. Otherwise the model would only learn training images and won't be able to properly create completely new ones.

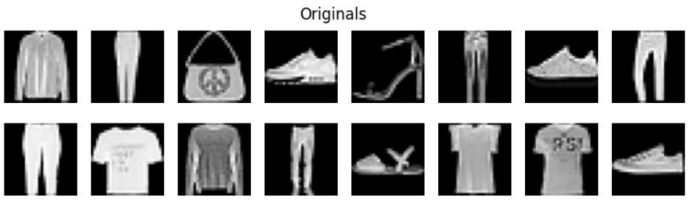

Figure 5. Sample of Fashion-MINST dataset

The generation shows that the model is able to generate new images from random vector in input, here are some example of image generation :

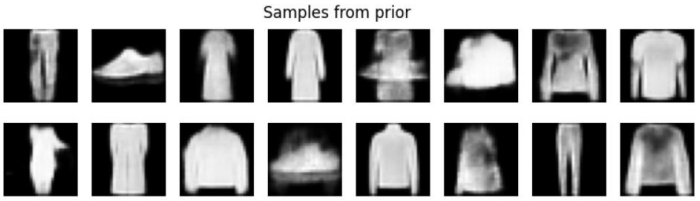

Figure 6. Sample generated by the VAE

As you can see the results are pretty plausible, it means that the VAE can determine high level features such as categories. Moreover, there are only limited signs of posterior collapse which indicate that the KL term is balanced.
Then, to improve the results the Conv-β-VAE has been implemented and the result of the loss prove that the model is converging to a minimum after few epochs.

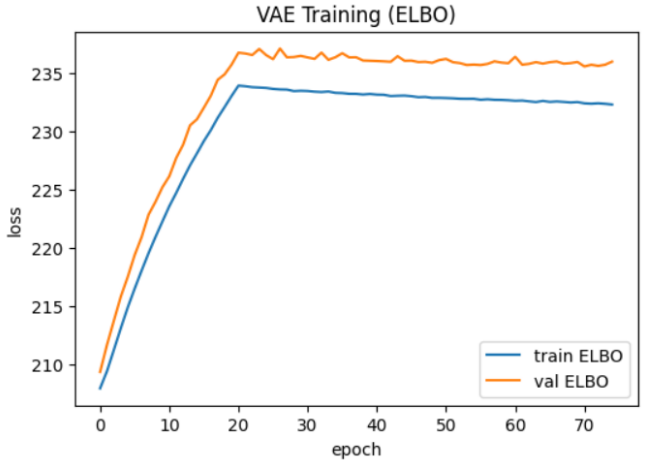

Figure 7. Conv-β-VAE training and validation loss based on epoch

The loss is increasing at the beginning because of the β-annealing. However, it seems to decrease and converge as expected.

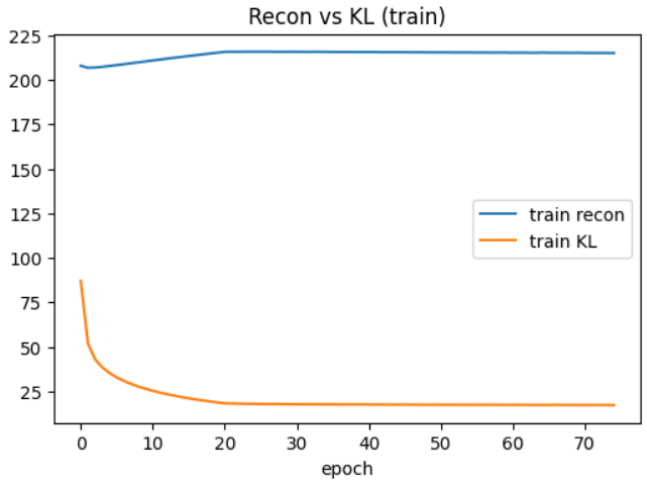

Figure 8. Conv-β-VAE reconstruction and KL divergence based on epoch

The KL divergence stabilizes after few epochs. The reconstruction loss seems to reach a value. This indicates that the model has converged to a balance between reconstruction and latent space regularization.

Here are some example of generation for the Conv-β-VAE :

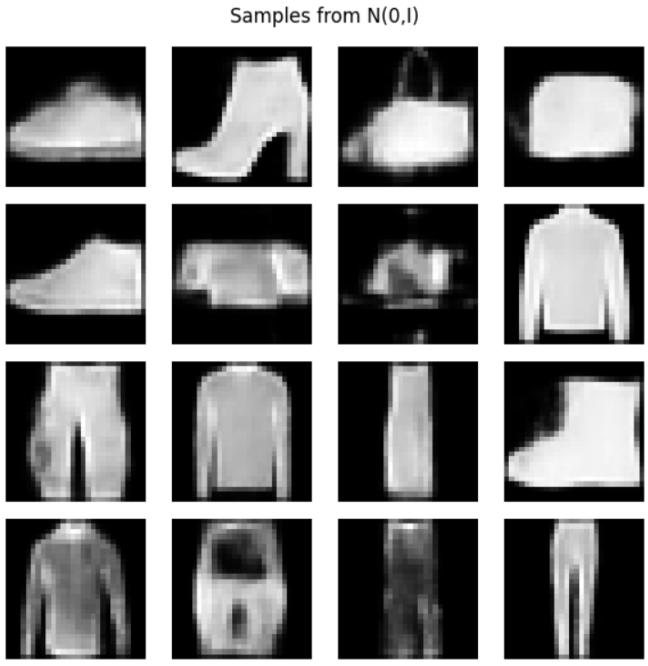

Figure 9. Images generated by the Conv-β-VAE after training

These examples looks even more realistic than the previous one showing that this model is more successful and the more important thing is that the images look sharper.

Latent Space Analysis

When we talk about VAE is important to mention the latent space analysis. In order to do that we can use the interpolation that gives us an hint of how much the latent space is well structured. If it reveals a smooth and continuous evolution of the reconstructed images demonstrates that the model has learned a well-structured latent manifold where vector which are closer has a similar image transformation. That confirms that the model effectively learn something and not only memorize the sample of the training set.

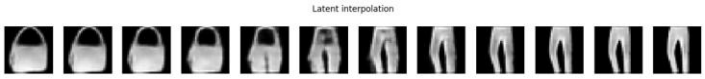

Figure 10. Latent interpolation between two sample of the database by the Conv-β-VAE

In this case we can be satisfied indeed intermediate samples has structural coherence and show gradual changes in shape and texture rather than abrupt transformations.

It means the KL divergence term and the β-annealing schedule successfully regularize the latent distribution. This is another confirmation that the model is well trained and there is no posterior collapse.

3.2 – GAN Training and Generated Sample Quality

From the grid search, v6_ttur_smooth obtained the lowest KID. This setup has been stabilized with Exponential Moving Average (EMA), achieving the final DCGAN + EMA model presented below.

Here a some example of generated images at the end of the training for the best model :

Figure 11. Samples generated by the DCGAN model after training

These examples seems realistic and for a qualitative analysis can be considered as good results. In order to improve the model we used Exponential Moving Average(EMA).

Figure 12. Samples generated by the DCGAN with EMA model after training

In this grid, the shape of the samples seems more regular. Indeed, items such as boots, skirts and tops seems to have smoother outlines. Edges appear sharper at the native 28×28 scale. Visually, there are fewer incoherent samples (like the second one from the top left corner). However almost every tile is interpretable as a plausible article of clothing. This is a convincing element of the improvement brought by EMA. Suppressing noisy updates produces more consistent and more faithful samples.

4 – Discuss

Both models produces both realistic samples. To evaluate the generative capabilities of both architectures, a qualitative and quantitative comparison was carried out between the VAE and the DCGAN. While both models aim to learn the underlying data distribution and produce realistic samples, their learning principles are completely different.

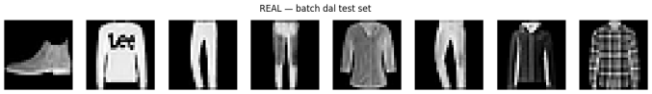

Figure 13. Samples from the Fashion-MINST dataset

The VAE explicitly learns to reconstruct input images, providing a direct measure of reconstruction fidelity (BCE). Samples reconstructed try to keep the general structure (according to the category) but implies a significant blurriness and loss of detail. This smoothing effect is caused by the probabilistic nature of the decoder, which tends to average multiple plausible outputs.

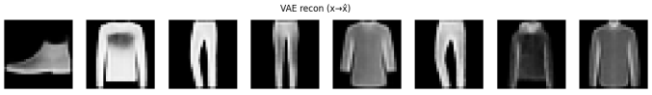

Figure 14. Samples reconstructed by the Conv-β-VAE

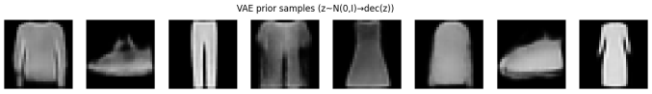

Figure 15. Samples generated by the Conv-β-VAE model after training

In contrast, the DCGAN does not reconstruct inputs but rather generates new samples from random latent vectors. DCGAN generated images looks sharper with high contrast and details particularly for shoes and bag categories. However, it might tend to create less diverse samples and visually similar images across the inputs.

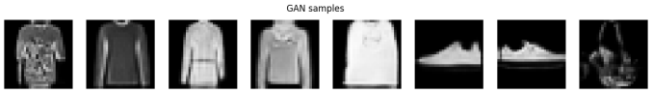

Figure 16. Samples generated by the DCGAN with EMA model after training

The latent space of the VAE is continuous and well-structured due to the KL regularization term and the β term adaptation. Whereas the DCGAN's latent space is not explicitly regularized. This leads the VAE to produce more realistic interpolation between samples with smooth transition. However the DCGAN produces highly realistic samples even if the interpolation are less coherent. Indeed it does not capture underlying semantic structures as good as the VAE.

The VAE training process is stable because it relies on single loss function that balances reconstruction and latent space regularization. Both reconstruction and KL divergences slowly converge and stabilize after few epochs.

Meanwhile, the GAN has a oscillatory loss behavior due to the adversarial interplay between generator and discriminator. A sweep of hyperparameter such as learning rates and techniques such as Exponential Moving Average (EMA) were necessary to achieve better convergence.

Visually the GAN outperform the VAE in term of image sharpness and details. Generated samples seems more realistic with high frequency information. Nevertheless, the VAE exhibits higher sample diversity and a better coverage of the data distribution.

To complement qualitative comparison with images, quantitative metrics were computed.

Three measures were used :

- Fréchet Inception Distance (**FID):** It compares the real and synthetic distribution with the aim of establishing how realistic the set of generated samples is. The lower the value, the closer the distributions are and the higher the realism is. The GAN achieved a lower FID than the VAE, confirming that GAN synthetic samples results more realistic then the VAE's ones. This is the final proof of what we said before from a qualitative POV.

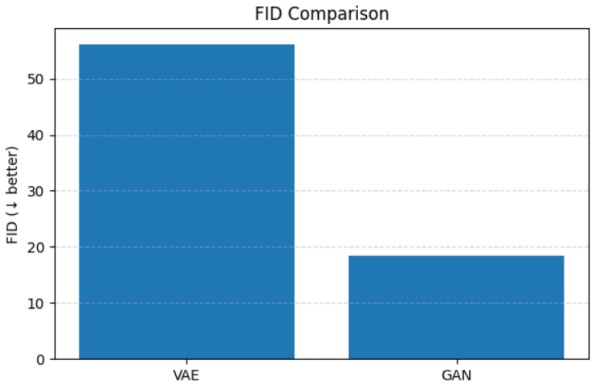

Figure 17. FID comparison between both models

- Multi-Scale Structural Similarity **(MIS-SSIM):** It evaluates the similarity between two generated sample, and it measures how repetitive the generated samples are. The lower the value, the higher the differences between samples are and the higher the diversity is. The GAN show better results also with this metrics indeed has a lower MS-SSIM value, indicating greater diversity among generated samples then VAE. So far the GAN produce more realistic and heterogeneous synthetic samples.

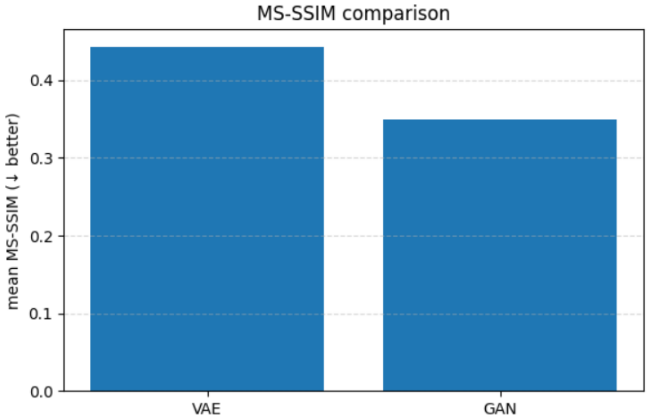

Figure 18. MS-SSIM comparison between both models

- **NN Memorization:** The metric measures how distant the generated samples are from the training samples. While the duplicate rate shows how many samples are a copy of one of the training samples (good indicator of overfitting). Both models showed very impressive duplication rates indeed both achieves the 0%, demonstrating that they generalize rather than memorize the training set. It proves that there is no overfitting. Talking about the NN-distance the graphs show that VAE slightly overperform GAN this time. This

means that VAE is able to produce more distant sample from the training sample. This can be interpreted like an higher level of creativity. However this is not enough to justify the other advantages that GAN has shown.

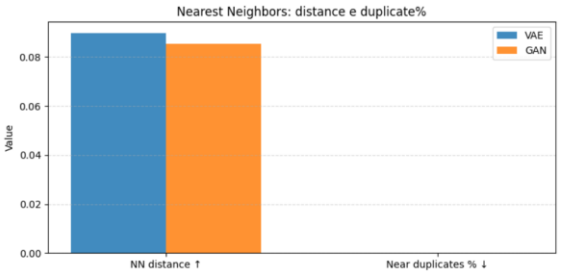

Figure 19. NN comparison between both models

These results resumed in the table below show how in term of image generation GAN overperform VAE in term of realism and variety; it underperform only in creativity but it still really close to VAE. However, the result of VAE are still valid like we see in the generated sample are quite realistic and only a little bit blurred.

Model	GAN	VAE
Metric		
FID ↓ (realism)	18.504083	56.165887
MS-SSIM ↓ (variety)	0.349014	0.443314
NN distance ↑ (creativity)	0.085464	0.089786
Near duplicates % ↓ (overfitting)	0.000000	0.000000

Figure 20. Values for each model of FID, MS-SSIM and NN

To conclude, both models successfully captured the distribution of Fashion-MNIST images, but with distinct advantages.

The VAE demonstrates smooth and semantically organized latent representations. This guarantees stable convergence and consistent sample generation. Although its quantitative metrics (FID and MS-SSIM) are less competitive than the GAN ones.

However, the synthetic samples are still visually coherent and meaningful; that is a confirmation of the model's ability to capture the global structure of the data even with lower visual detail and realism.

GAN demonstrates itself to be the best tested model achieving significantly better realism and sharper visual details. All these conclusions are supported by the metrics score. However, this comes with a higher training complexity and slightly reduced sample diversity, as indicated by the MS-SSIM and NN-distance values.

Limitations and Future Work

Both models achieve good performances but some aspect could be improved in future analysis. The current implementation uses the Fashion-MNIST dataset, that is a low resolution and grayscale datasets.

Evaluates the same architecture on other datasets could help to verify the model generation ability on a larger scale [3].

The model architecture also can be enhanced:

- The GAN network could benefit from alternative loss formulations such as the Wasserstein objective (WGAN-GP), or from the inclusion of spectral normalization to improve training stability [2].
- The β-VAE could be enhanced by introducing *skip connections* or *perceptual losses* to obtain sharper reconstructions.
- Otherwise other future works should compare the model with more advanced ones like StyleGAN2 or ResNet-VAE. This could be interesting in order to see how architectural depth and complexity influence the balance between realism and disentanglement [1].

References :

1. **He et al., 2016 – "Deep Residual Learning for Image Recognition"**, *CVPR 2016*
2. **Arjovsky et al., 2017 – "Wasserstein GAN"**, *ICML 2017*
3. **Sajjadi et al., 2018 – "Assessing Generative Models via Precision and Recall"**, *NeurIPS 2018*
4. **Kingma & Welling (2014)** – *Auto-Encoding Variational Bayes*, *ICLR 2014*
5. **Higgins et al. (2017)** – *β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework*, *ICLR 2017*
6. **Goodfellow et al. (2014)** – *Generative Adversarial Nets*, *NeurIPS 2014*
7. **Radford, Metz & Chintala (2016)** – *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks (DCGAN)*, *ICLR 2016*
8. https://medium.com/@rushikesh.shende/autoencoders-variational-autoencoders-vae-and-

%CE%B2-vae-ceba9998773d by Rushikesh Shende

9. https://developers.google.com/machine-learning/gan/gan_structure?hl=fr by Google Developers
10. **Hou, X., Shen, L., Sun, K., & Qiu, G. (2017).** ***Deep Feature Consistent Variational Autoencoder.*** *arXiv preprint arXiv:1610.00291.* https://arxiv.org/abs/1610.00291
11. Jason Brownlee. A Gentle Introduction to BigGAN – How It Works and Why It Matt ers.Machine Learning Mastery, 2021. https://machinelearningmastery.com/a-gentle-introduction-to-the-biggan
12. Karras, T., Aila, T., Laine, S., & Lehtinen, J. (2017). Progressive Growing of GANs f or Improved Quality, Stability, and Variati on. https://arxiv.org/pdf/1710.10196
13. Brock, A., Donahue, J., & Simonyan, K. (2 019). Large Scale GAN Training for High Fidelity Natural Image Synthesis (BigGAN).ICLR 2019.
14. Karras, T., Laine, S., & Aila, T. (2019). A Style-
    Based Generator Architecture for Generati ve Adversarial Networks (StyleGAN).CVP R 2019.https://arxiv.org/pdf/1812.04948